



CSE 711 - Compilers

Lecture 3 (Part 4)

Syntax Analysis

Ms. Samia Muntaha

Lecturer,

Department of Computer Science and Engineering
University of Chittagong

Outline

- ▶ Bottom-Up Parsing
- ▶ LR(I) items
- ▶ CLR Parser
- ▶ LALR parser
- ▶ Conflicts in CLR and LALR parsers
- ▶ Operator Precedence Parsers

LR(1) Items

$$I_n$$

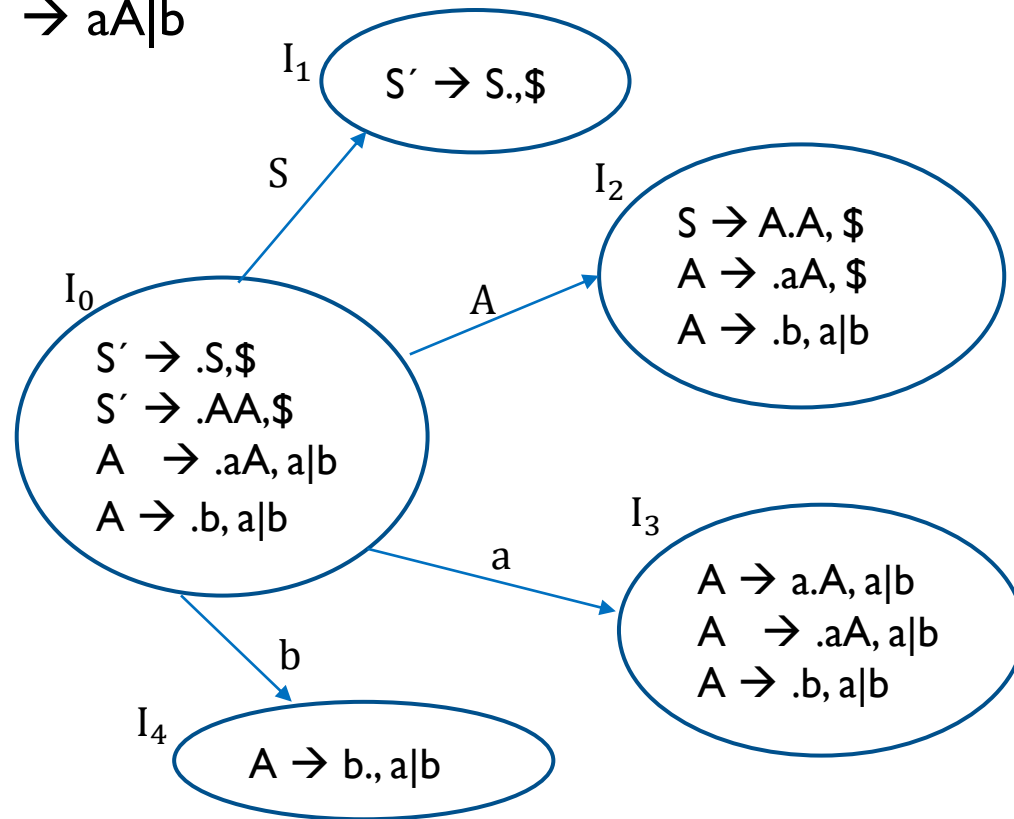
$S \rightarrow aA., a|b$

For LR(1) items, the rule numbers are placed under the lookaheads in the action table.

States	Actions			
	a	b	c	\$
n	r ₂	r ₂		

Canonical Collection of LR(1) Items

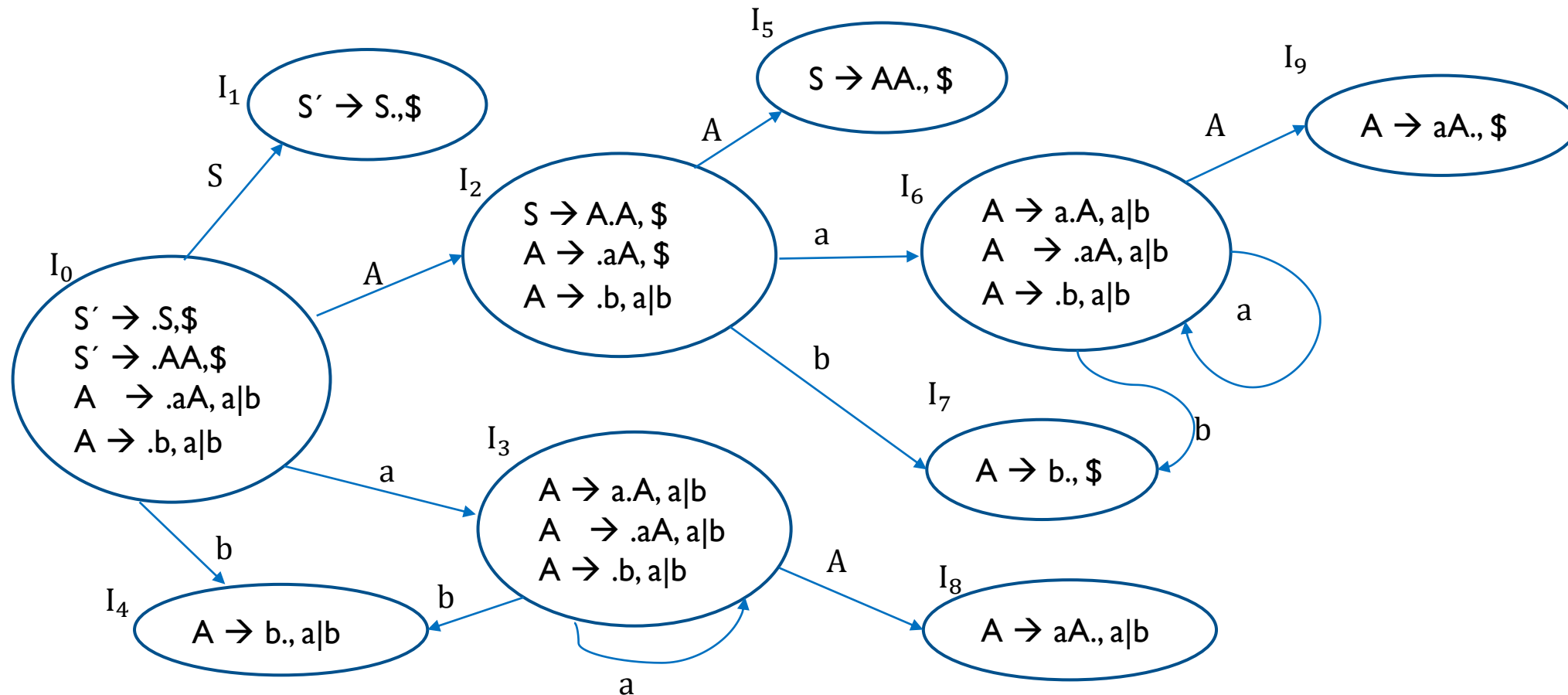
- ▶ $S' \rightarrow S$
- ▶ $S' \rightarrow AA$
- ▶ $A \rightarrow aA|b$



Rules:

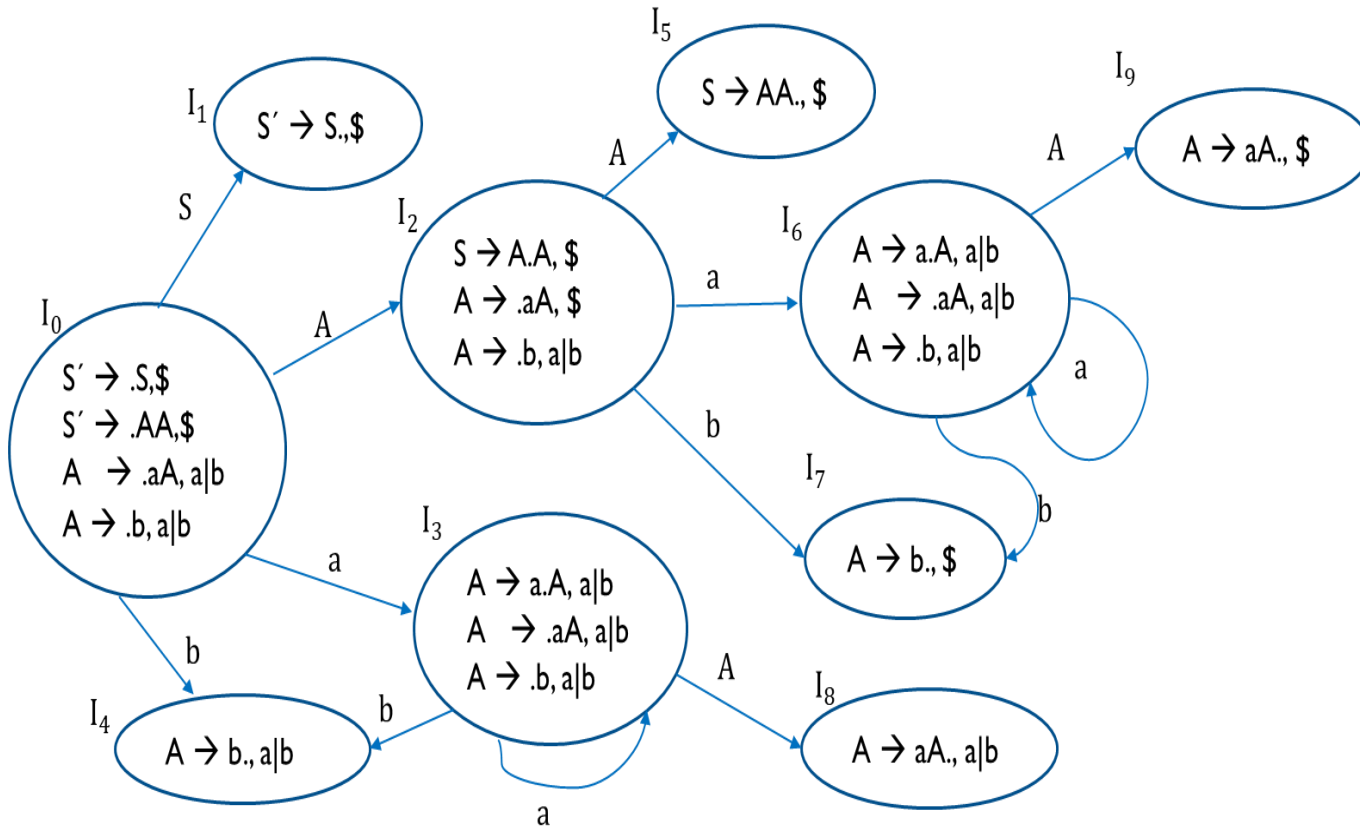
1. For Start symbol's production, the lookahead is '\$'.
2. In the closure set, the lookaheads of the LR(1) items are determined based on the rest of the items' FIRST set.
3. Lookahead will never change during transitions, however, may change within state while determining closure set.

Canonical Collection of LR(1) Items



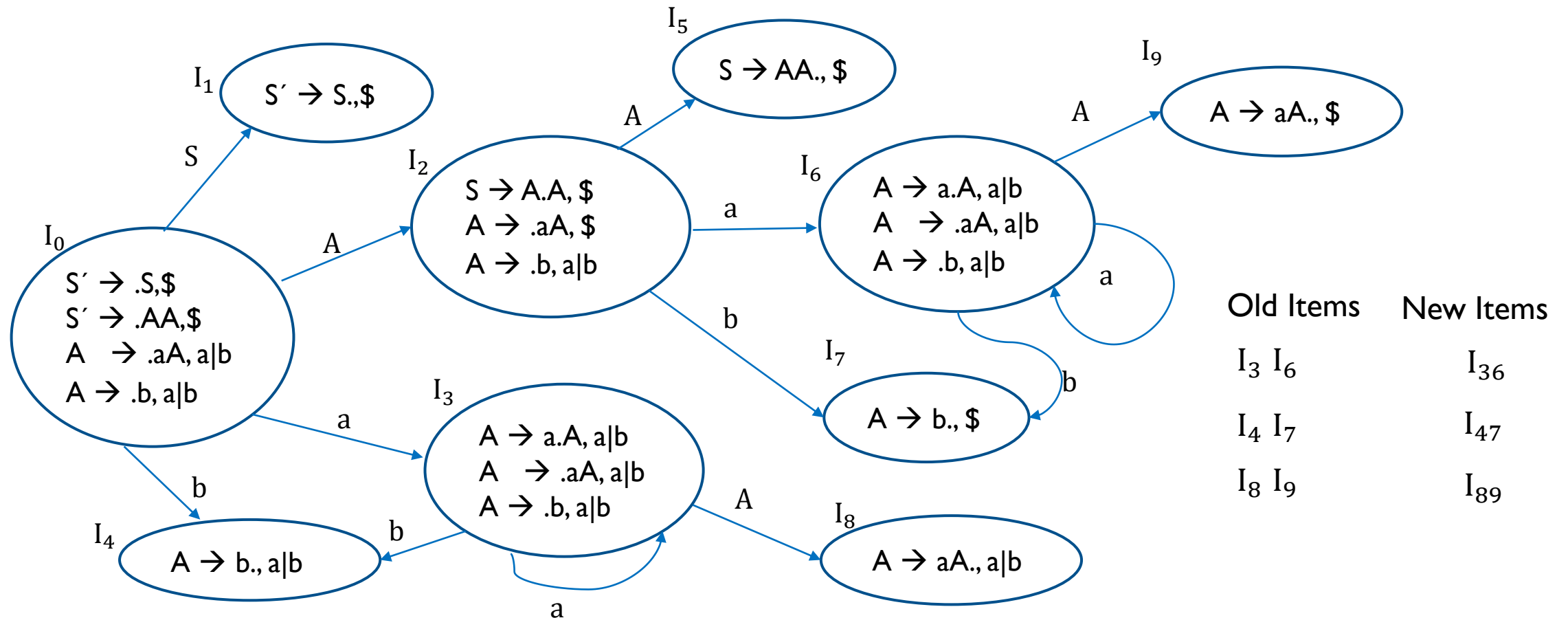
CLR(1) Parser

$S' \rightarrow S$
 $S' \rightarrow AA$
 $A \rightarrow aA|b$



States	Actions			GOTO	
	a	b	\$	A	S
0	S_3	S_4		2	1
1			Accept		
2	S_6	S_7		5	
3	S_3	S_4		8	
4	r_3	r_3			
5			r_1		
6	S_6	S_7		9	
7			r_3		
8	r_2	r_2			
9			r_2		

LALR(1) Parser



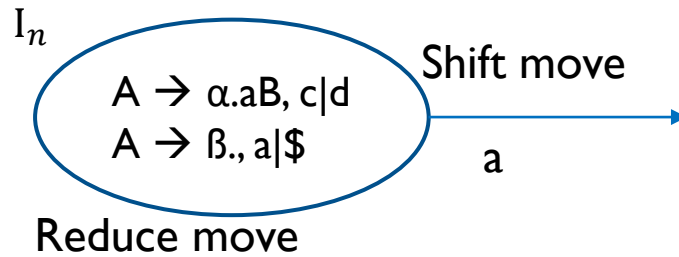
Conversion to LALR(1) Parsing Table

States	Actions			GOTO	
	a	b	\$	A	S
0	S ₃₆	S ₄₇		2	1
1			Accept		
2	S ₃₆	S ₄₇		5	
36	S ₃₆	S ₄₇		89	
47	r ₃	r ₃	r ₃		
5			r ₁		
89	r ₂	r ₂	r ₂		

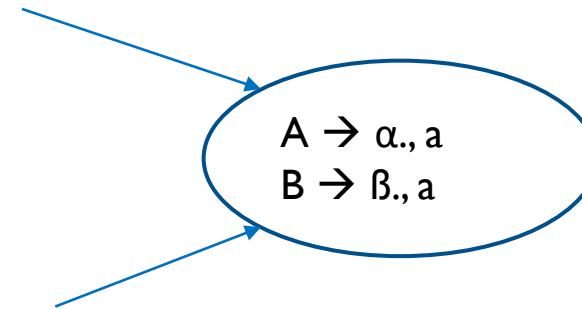
Old Items	New Items
I ₃ I ₆	I ₃₆
I ₄ I ₇	I ₄₇
I ₈ I ₉	I ₈₉

Conflicts in CLR(1) and LALR(1) Parser

► Shift-reduce Conflict



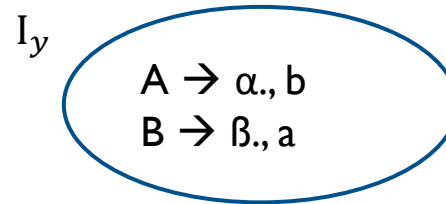
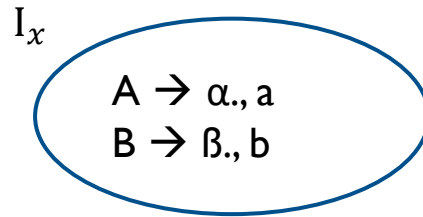
► Reduce-reduce Conflict



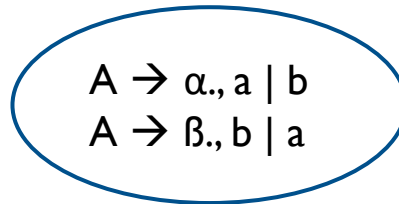
Not CLR(1)
Not LALR(1)

Conflicts in CLR(1) and LALR(1) Parser

► Reduce-Reduce Conflict



No conflict for CLR(1)



Conflict for LALR(1)

No S-R Conflict for CLR(1) $\rightarrow$ No S-R Conflict in LALR(a)
No R-R Conflict for CLR(1) $\neq$ No R-R Conflict in LALR(a)

LR(1) & LALR

- LR(1) automata are identical to LR(0) except for the “items” that make up the states
- LR(0) items:
 $X ::= s1 . s2$
- LR(1) items
 $X ::= s1 . s2, T$
• Idea: sequence $s1$ is on stack; input stream is $s2 T$
• Find closure with respect to $X ::= s1 . Y s2, T$ by adding all items $Y ::= s3, U$ when $Y ::= s3$ is a rule and U is in $First(s2 T)$
• Two states are different if they contain the same rules but the rules have different look-ahead symbols
 - Leads to many states
 - LALR(1) = LR(1) where states that are identical aside from look-ahead symbols have been merged
 - ML-Yacc & most parser generators use LALR
- READ: Appel 3.3 (and also all of the rest of chapter 3)

Operator Precedence Parsers

- ▶ It is used for **mathematical expressions**
- ▶ It processes **operator grammars**
- ▶ It can handle **ambiguous grammars**

Operator Grammars are used to define Operators.

Restrictions:

1. No adjacent non-terminals.
2. No epsilon productions.

$E \rightarrow EAE \mid id$

$A \rightarrow + \mid *$



$E \rightarrow E+E \mid E * E \mid id$

Conversion to Operator Grammar

- ▶ $S \rightarrow SAS \mid a$
- ▶ $A \rightarrow bSb \mid b$

$$S \rightarrow SbSbS \mid SbS \mid a$$

Construction of Operator Relation Table

$E \rightarrow E + E \mid E * E \mid id$

Ambiguous Grammar!

- ▶ $id \rightarrow$ Highest Precedence
- ▶ $\$ \rightarrow$ Lowest Precedence

Precedence	Associativity
$*, /, \%$	Left to right
$+, -$	Left to right

	id	+	*	\$
id	--	>	>	>
+	<	>	<	>
*	<	>	>	>
\$	<	<	<	

Operator Precedence Parsing

$E \rightarrow E+E \mid E * E \mid id$

- ▶ If $TOS < LookAhead$, PUSH(Shift) LookAhead in Stack.
- ▶ If $TOS > LookAhead$, POP(Reduce).

id + id * id \$

id
\$

	id	+	*	\$
id	--	>	>	>
+	<	>	<	>
*	<	>	>	>
\$	<	<	<	

Operator Precedence Parsing

$E \rightarrow E + E \mid E * E \mid \text{id}$

- ▶ If $\text{TOS} < \text{LookAhead}$, PUSH(Shift) LookAhead in Stack.
- ▶ If $\text{TOS} > \text{LookAhead}$, POP(Reduce).

id + id * id \$

id
+
\$

	id	+	*	\$
id	--	>	>	>
+	<	>	<	>
*	<	>	>	>
\$	<	<	<	

Operator Precedence Parsing

$E \rightarrow E+E \mid E * E \mid id$

- ▶ If $TOS < LookAhead$, PUSH(Shift) LookAhead in Stack.
- ▶ If $TOS > LookAhead$, POP(Reduce).

id + id * id \$

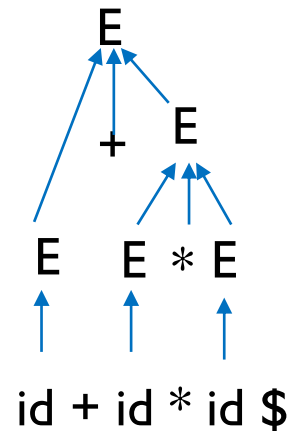
*
+
\$

	id	+	*	\$
id	--	>	>	>
+	<	>	<	>
*	<	>	>	>
\$	<	<	<	

Operator Precedence Parsing

$E \rightarrow E + E \mid E * E \mid id$

- ▶ If $TOS < LookAhead$,
PUSH(Shift) LookAhead in Stack.
- ▶ If $TOS > LookAhead$,
POP(Reduce).



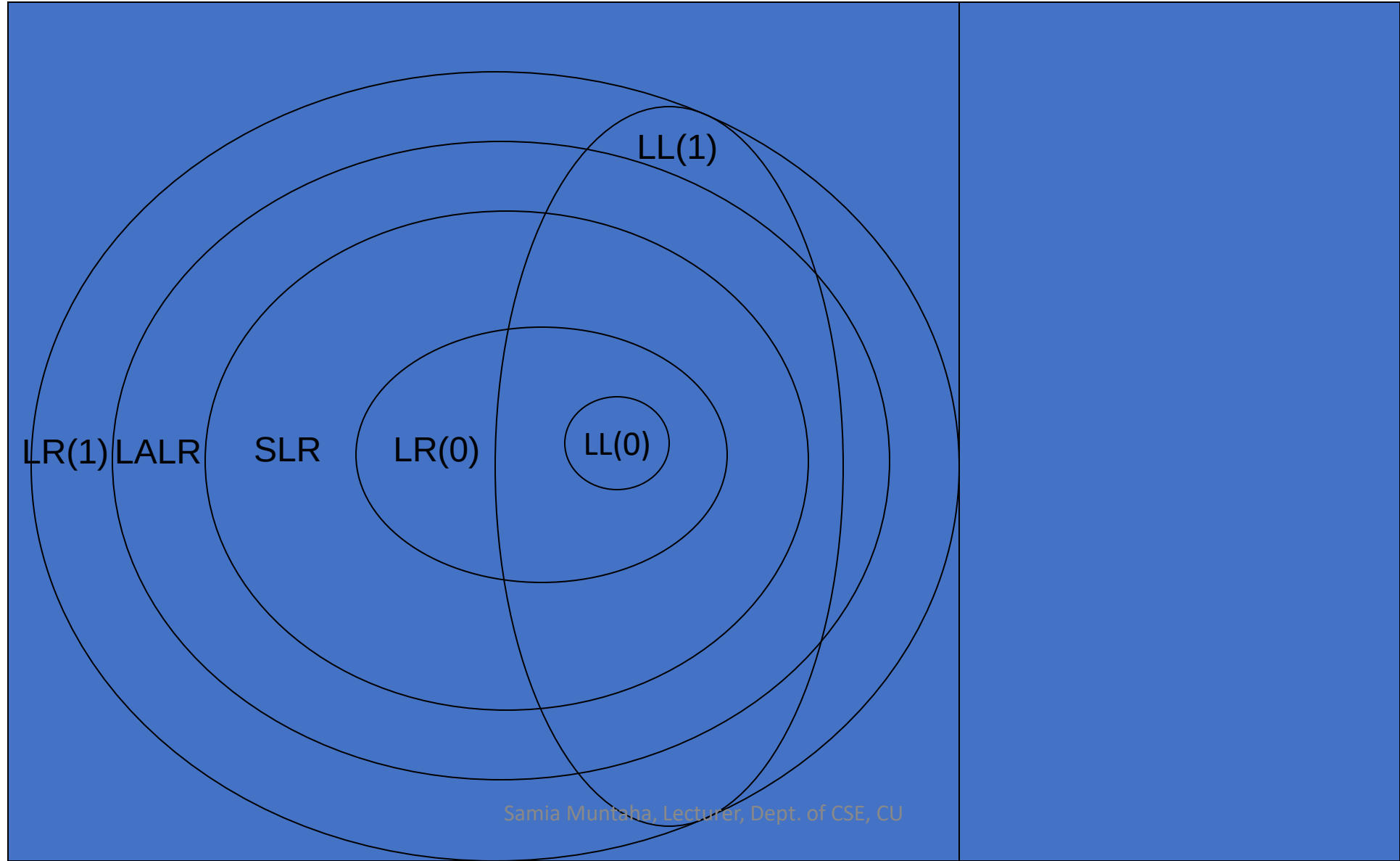
id
*
+
\$

	id	+	*	\$
id	--	>	>	>
+	<	>	<	>
*	<	>	>	>
\$	<	<	<	

Grammar Relationships

Unambiguous Grammars

Ambiguous Grammars



Summary

- LR parsing is more powerful than LL parsing, given the same look ahead
- To construct an LR parser, it is necessary to compute an LR parser table
- The LR parser table represents a finite automaton that walks over the parser stack
- ML-Yacc uses LALR, a compact variant of LR(1)

Assignment

- The grammar for if and if-else statements is

$$S \rightarrow \text{if } (E) S$$
$$| \text{if } (E) S \text{ else } S$$
$$| \text{id} = \text{num}$$
$$E \rightarrow \text{id} == \text{num}$$

where S is a statement and E is an expression (boolean).

- Write the action and goto tables for this grammar.
- Find the shift/reduce conflict(s) and decide how to handle them.